

ECE 382N-Sec (FA26):

L7: Hardware Root of Trust and Secure Coprocessors

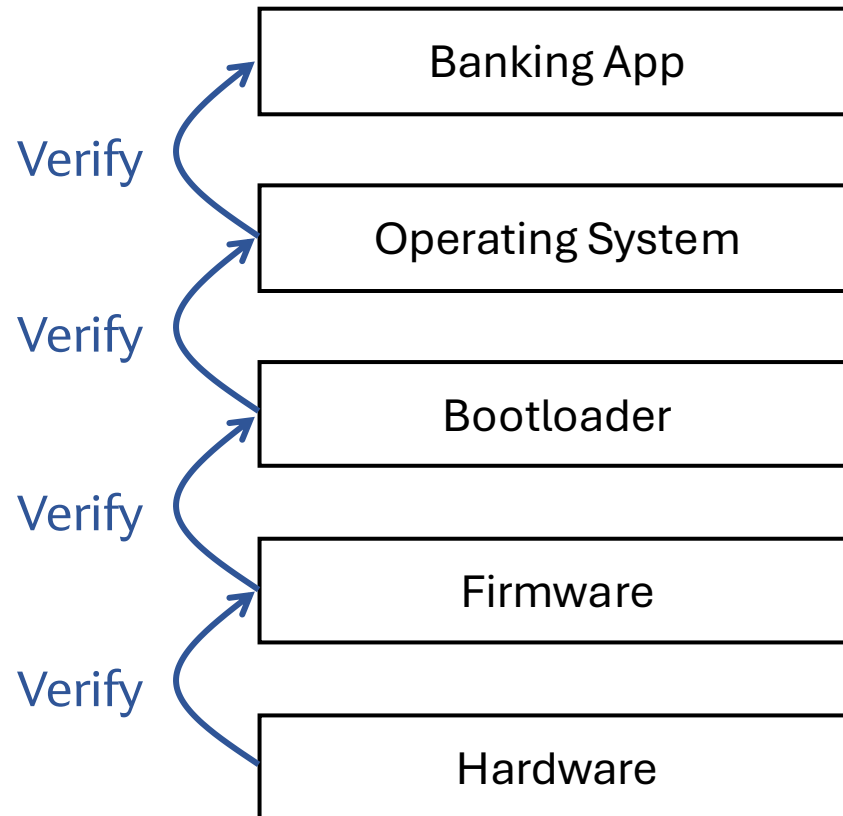
Neil Zhao

neil.zhao@utexas.edu

Before We Start

- Trusted computing often involves cryptography
- This course focuses on general crypto primitives instead of specific algorithms and their implementations
 - These primitives are nice “hammers” to system builders
 - How these hammers are built is fascinating, but it’s out-of-scope for this course
- Our discussion simplifies certain aspects of these crypto primitives. It is good for building an intuitive understanding, but please do consult and follow various crypto standards for anything serious. Don’t re-invent the hammer!
- **A good reference:** “Serious Cryptography: A Practical Introduction to Modern Encryption” by Jean-Philippe Aumasson

Why Do We Trust?



We trust the bank app because the OS verified the app's integrity

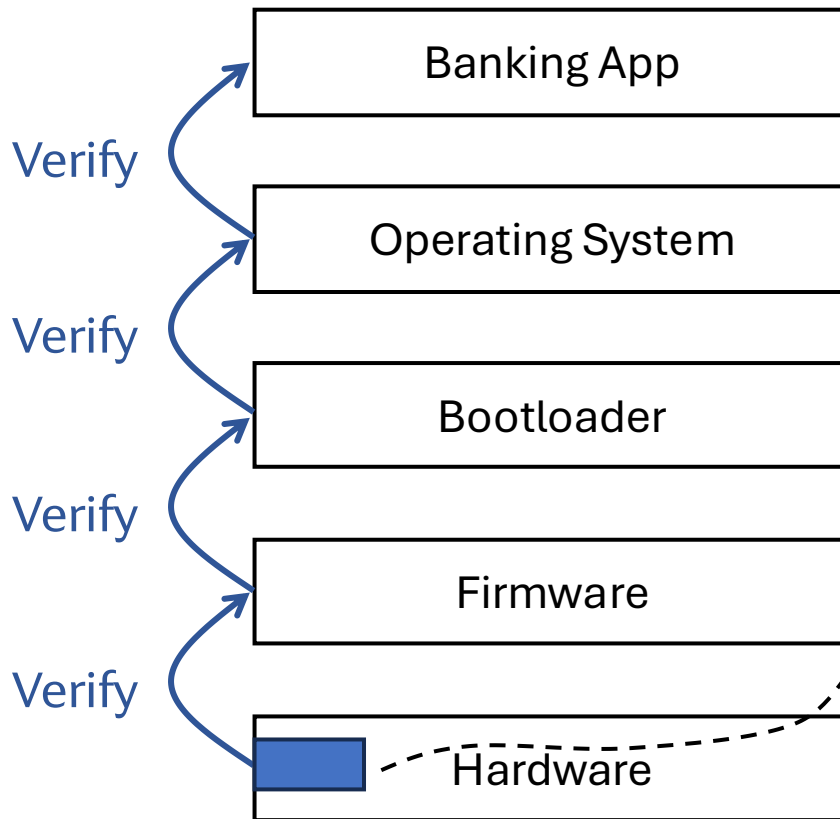
We trust the OS because the bootloader verified the OS's integrity

We trust the bootloader because the firmware verified the bootloader's integrity

We trust the firmware because the hardware verified the firmware's integrity

We trust the hardware because ...

Hardware Root of Trust



Typical form: Hardware starts execution from read-only memory (ROM), which contains information and instructions used to validate the firmware

Justifying the trust:

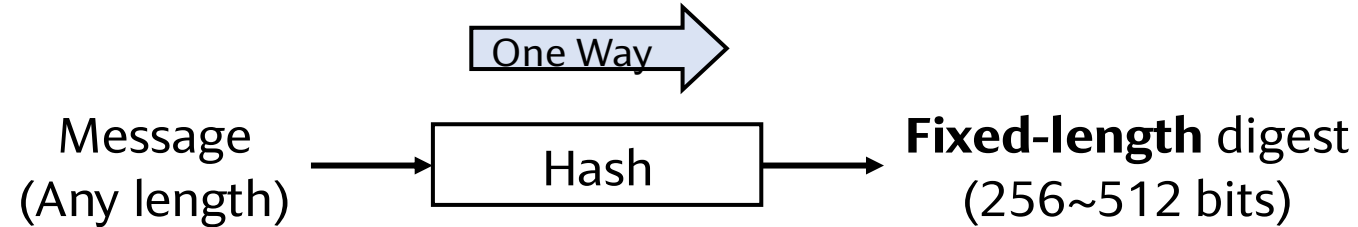
ROM is programmed during chip manufacturing and is often baked into the physical circuit

⇒ Not writable by the software

⇒ Hard to be physically tampered with

(Hardware) Root of Trust → **Integrity** is inherently trusted

Hammer 1: Cryptographic Hash Functions



Avalanche effect (example uses SHA-256):

"44 students are registered for ECE-382N" → "f1246ddddd902aa8de27ddce24bd24031..."

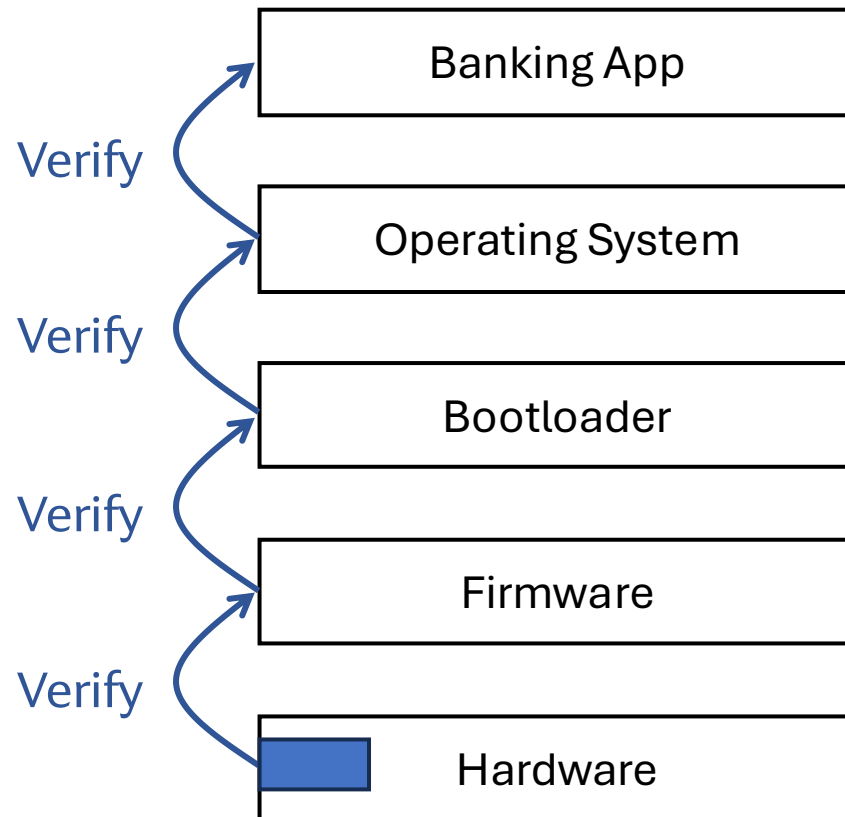
"43 students are registered for ECE-382N" → "4bb1db3619a8442661fc9107b1767483..."

Security properties:

- **Preimage resistance:** For any random hash value h , it's practically impossible to find a message M such that $\text{Hash}(M) = h$ in practice
- **Collision resistance:** Despite the inevitability of collision, it's practically impossible find two distinct messages that hash to the same value

Note: Not all hash functions are cryptographic hash functions! E.g., Cyclic redundancy checks (CRCs) are not cryptographic hashes

Bootstrapping the Trust



Hardware ROT:

- Hash the firmware
- Compare the digest against the burnt-in **reference value**
- Match?
 - Yes $\Rightarrow$ Execute the firmware and enter the next stage
 - No $\Rightarrow$ Abort!

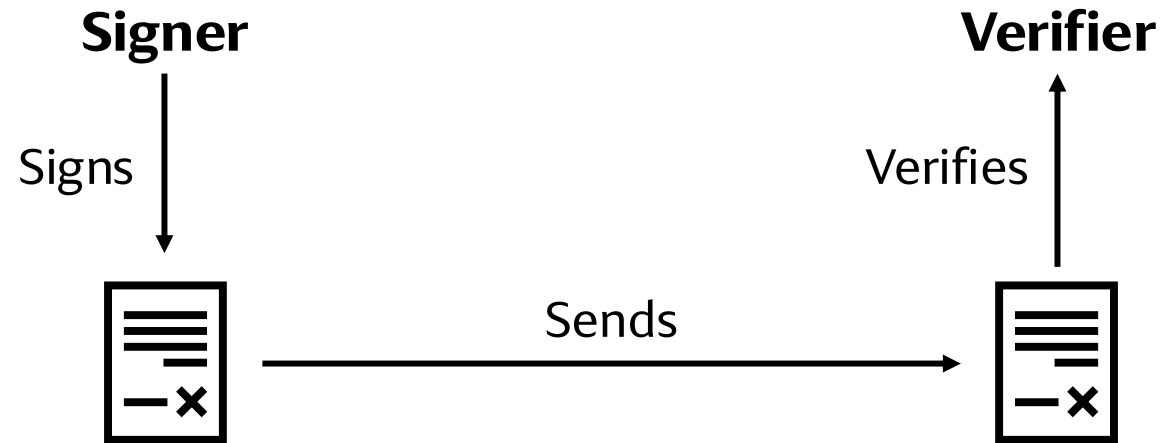
Firmware:

- Hash the bootloader
- ... (It's recursive)

Limitation: Won't work after firmware/OS updates

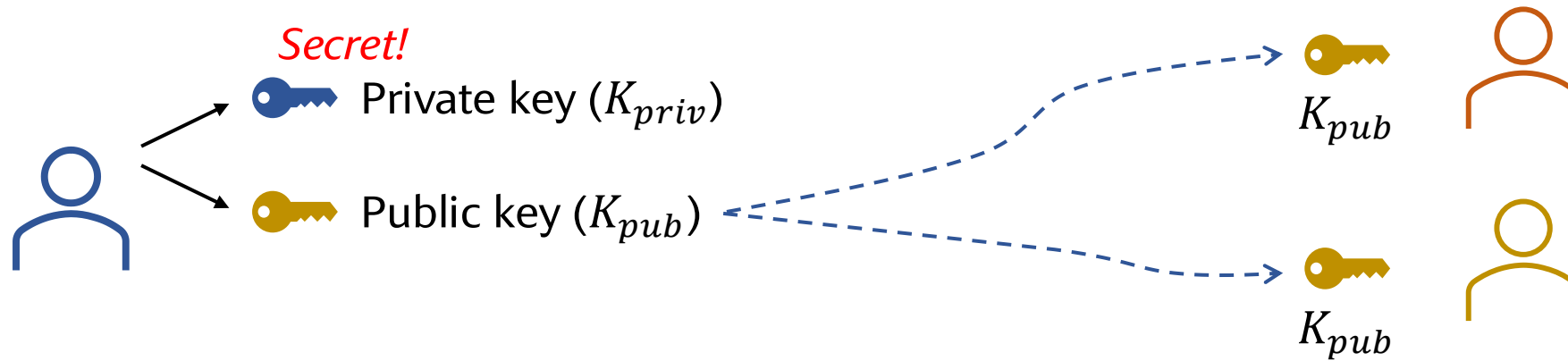
More Practical Policy: We accept firmware/bootloader/... from vendors we trust. But how?

Hammer 2: Digital Signature



How to achieve this?

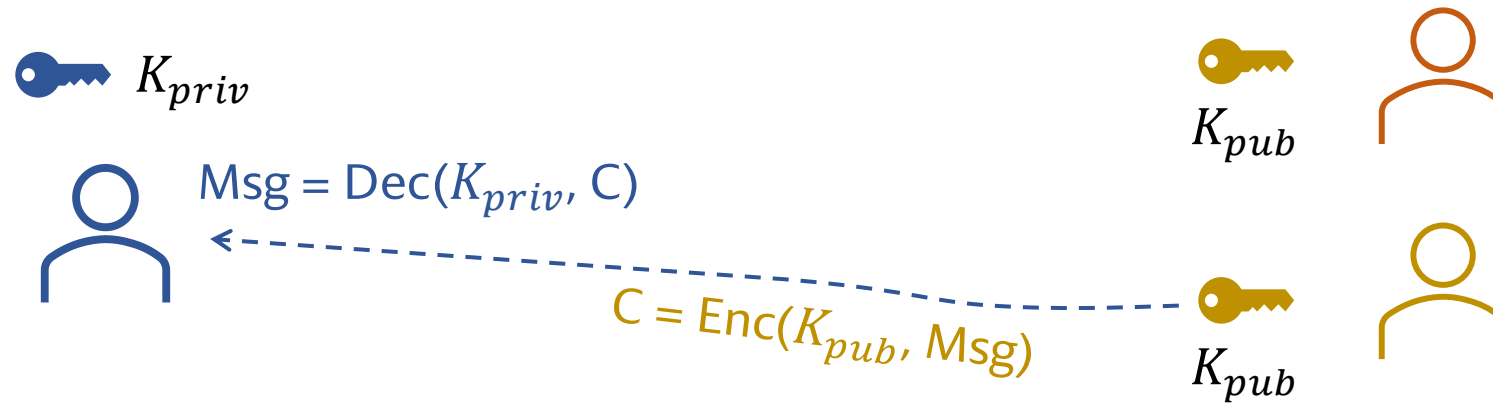
Hammer 3: Asymmetric Crypto



Example: RSA

Property: Messages encrypted with K_{priv} can only be decrypted with K_{pub}

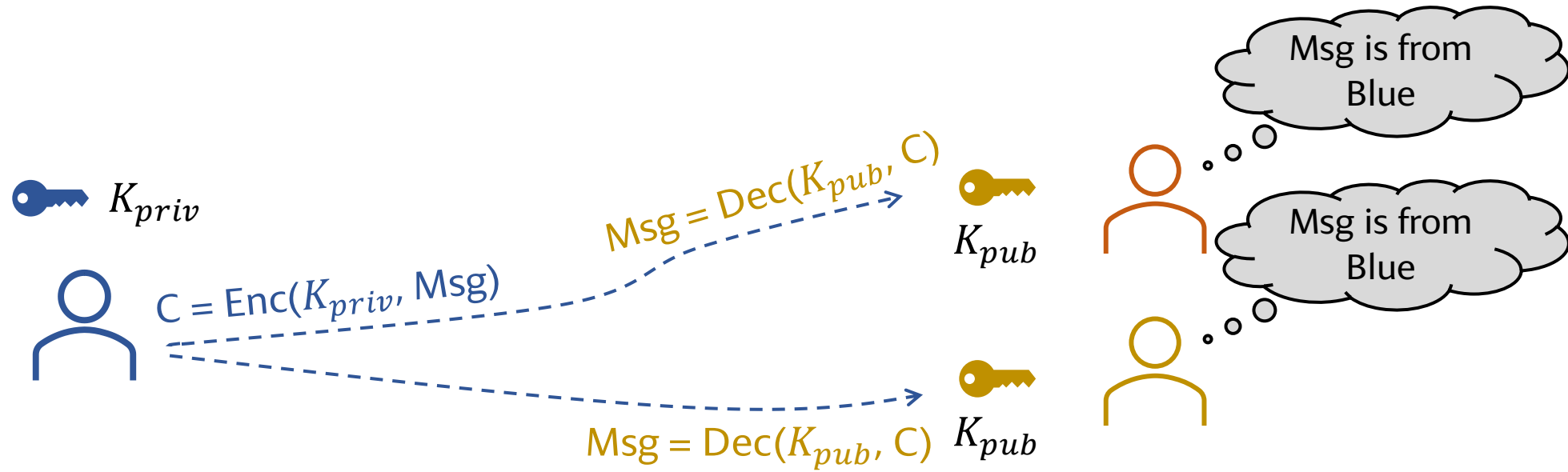
Hammer 3: Asymmetric Crypto



Example: RSA

Property: Messages encrypted with K_{priv} can only be decrypted with K_{pub}

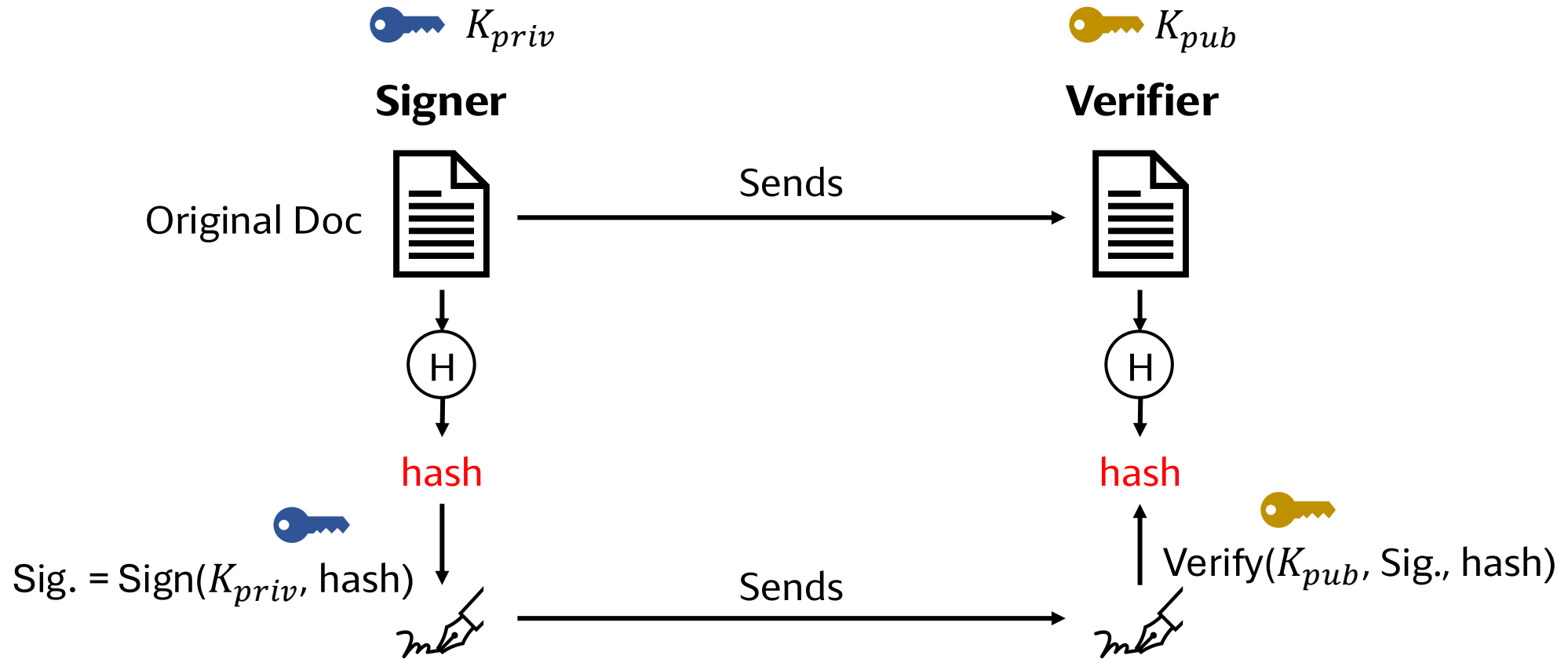
Hammer 3: Asymmetric Crypto



Example: RSA

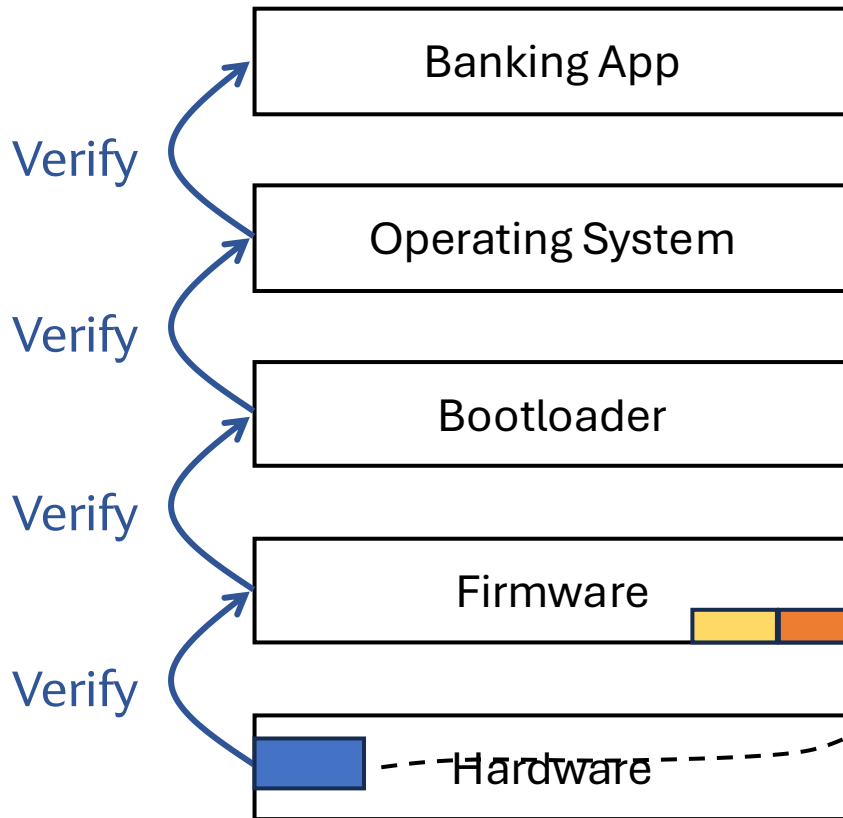
Property: Messages encrypted with K_{priv} can only be decrypted with K_{pub}

Digital Signature*



*Actual schemes are more complex than this

A More Flexible Way of Bootstrapping the Trust



Hardware ROT:

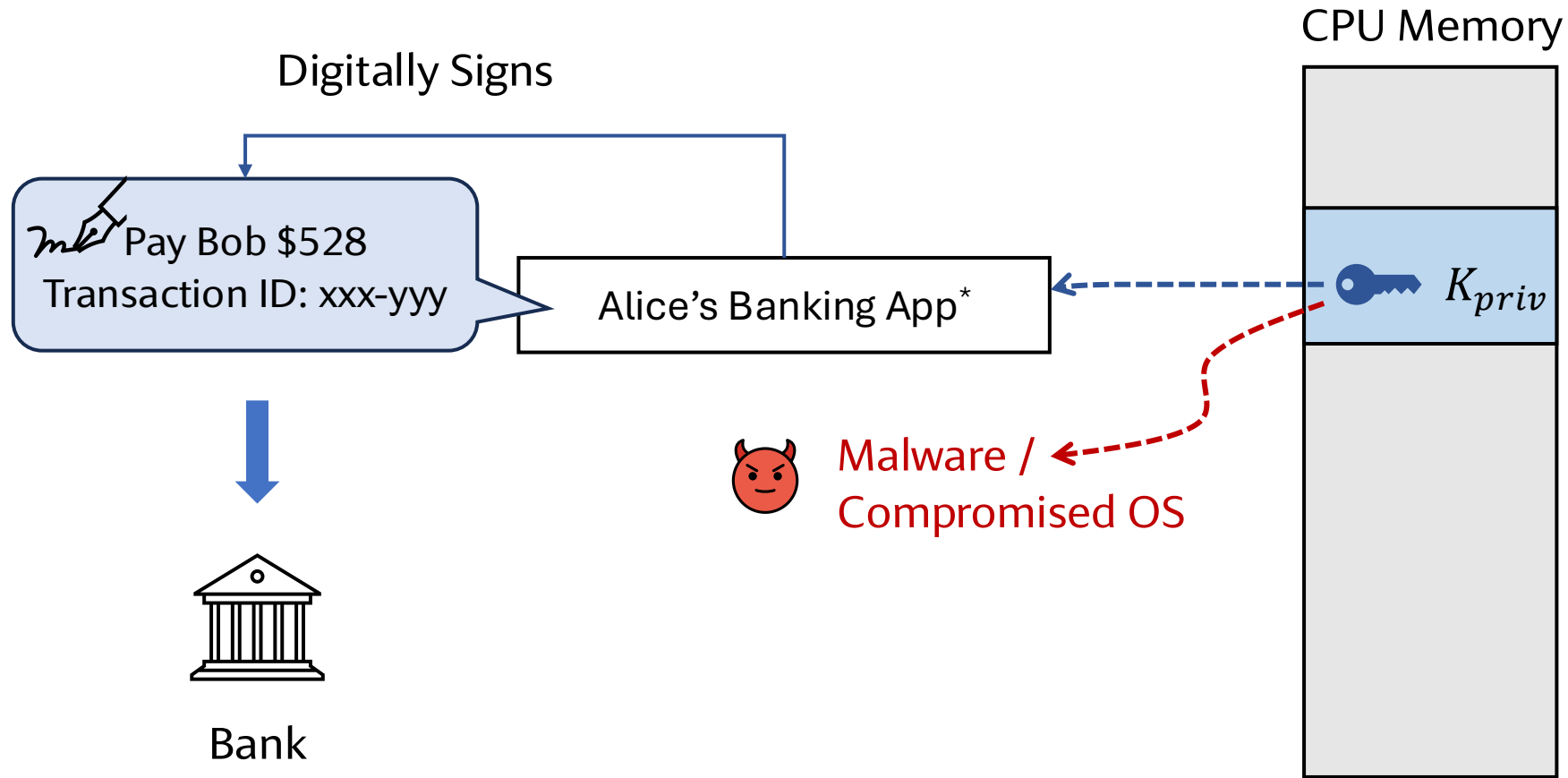
- Do I see a public key I trust?
- Can this signature be verified using this public key?

→ Carries the vendor's public key and the signature of the firmware

→ Stores public keys of the vendors we trust*

*We usually store the hash digests of these public keys

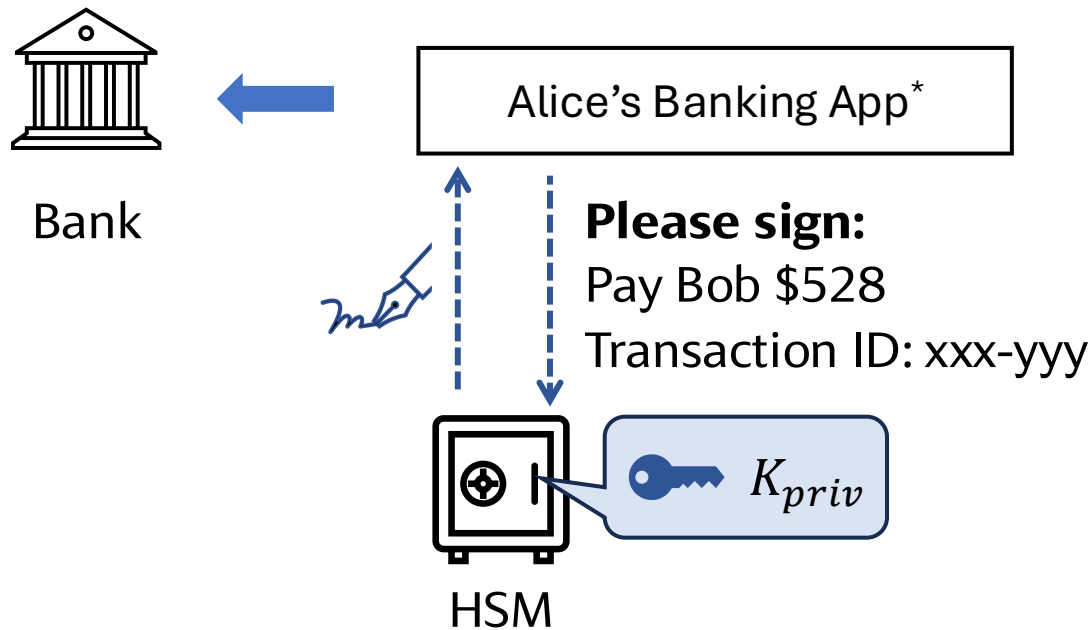
After the Trust is Established



*Just an illustration, not a real scheme

Store the Secret in a Hardware Security Module (HSM)

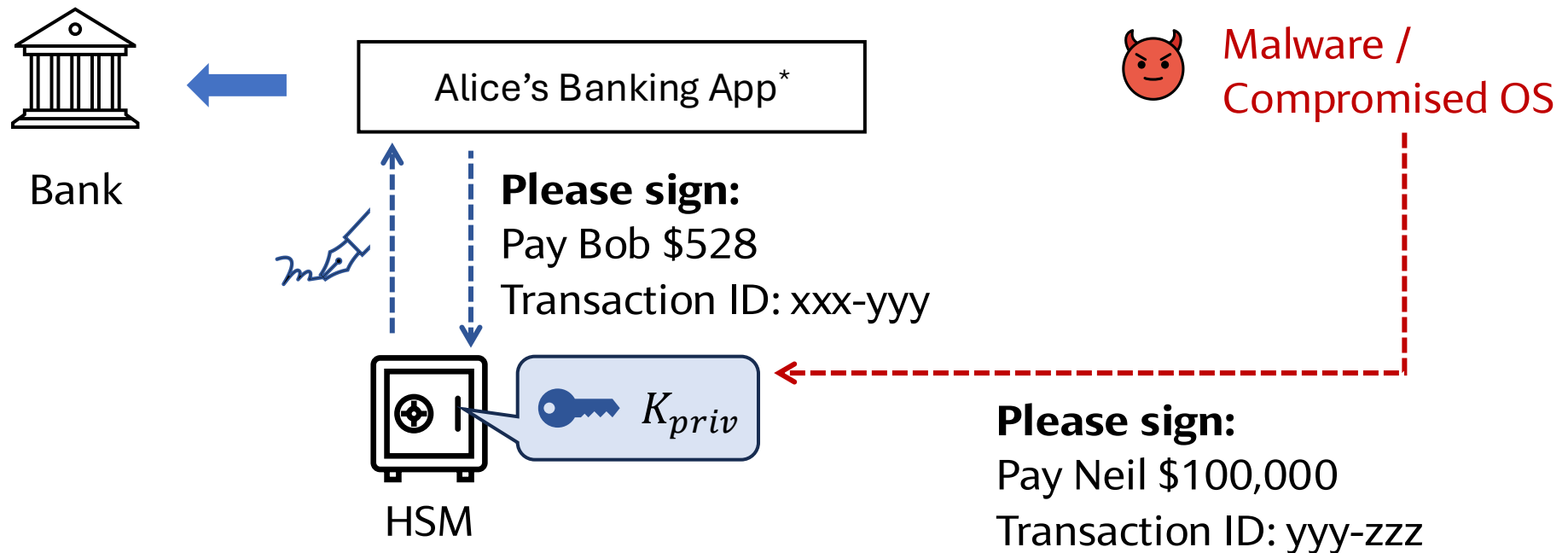
The secret private key does not need to be exposed to the software



*Just an illustration, not a real scheme

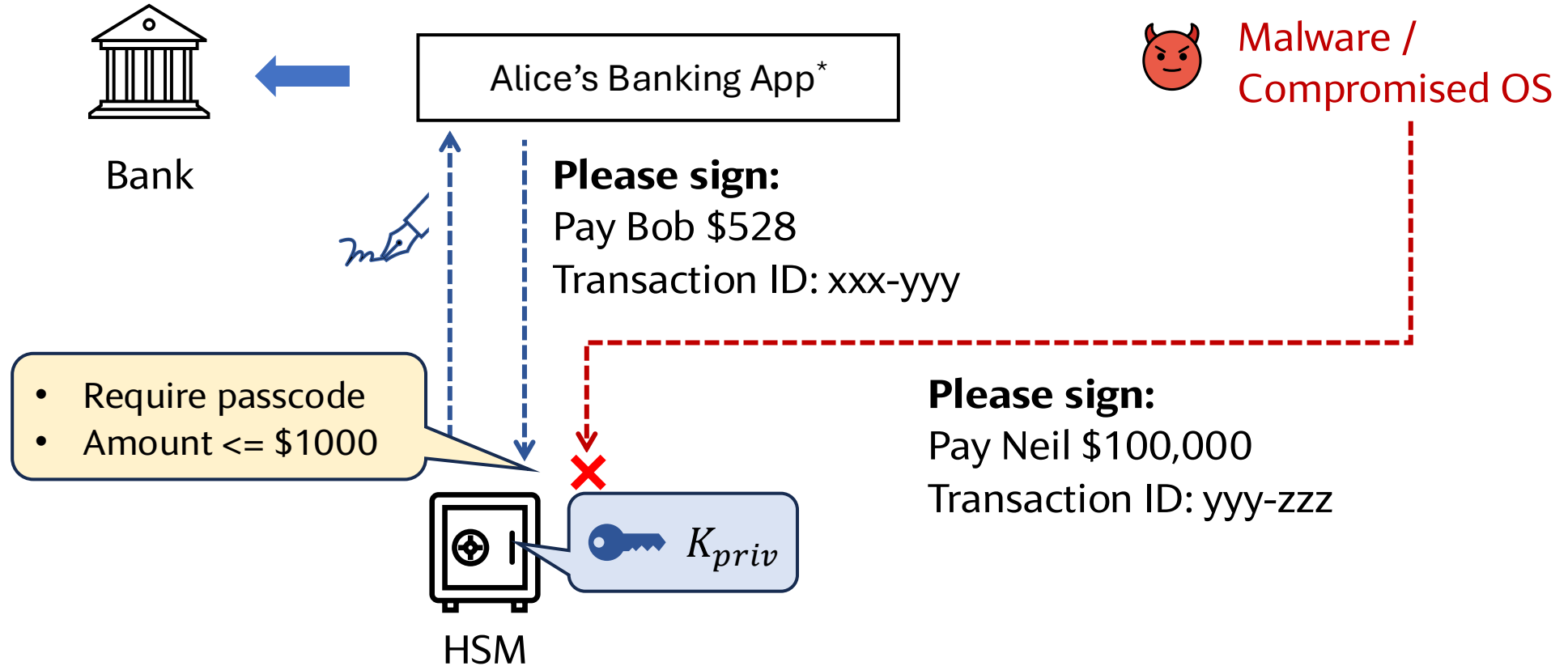
Store the Secret in a Hardware Security Module (HSM)

Blocking direct key access is not enough! We also need policy restricting interactions with HSM



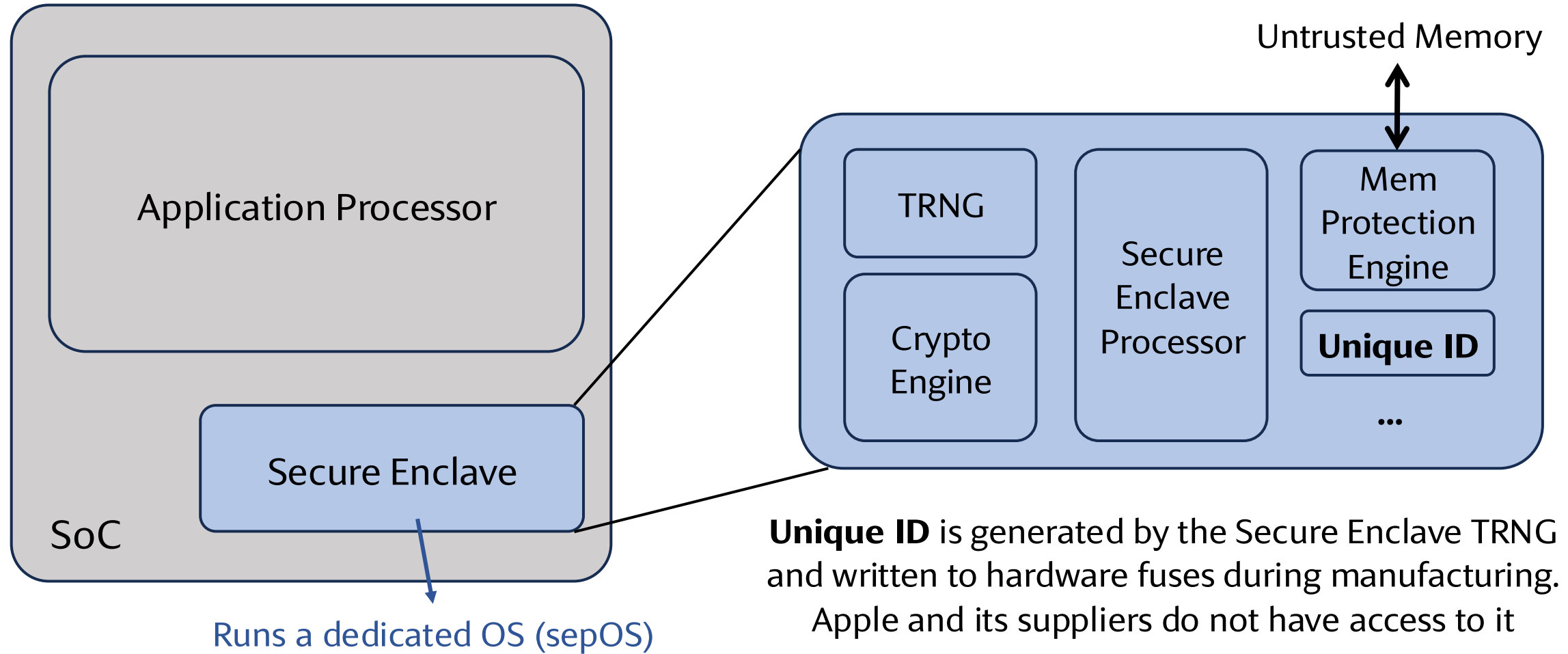
*Just an illustration, not a real scheme

Restricting Interactions with HSM



*Just an illustration, not a real scheme

Apple Secure Enclave



Example: Passcode-Protected Storage



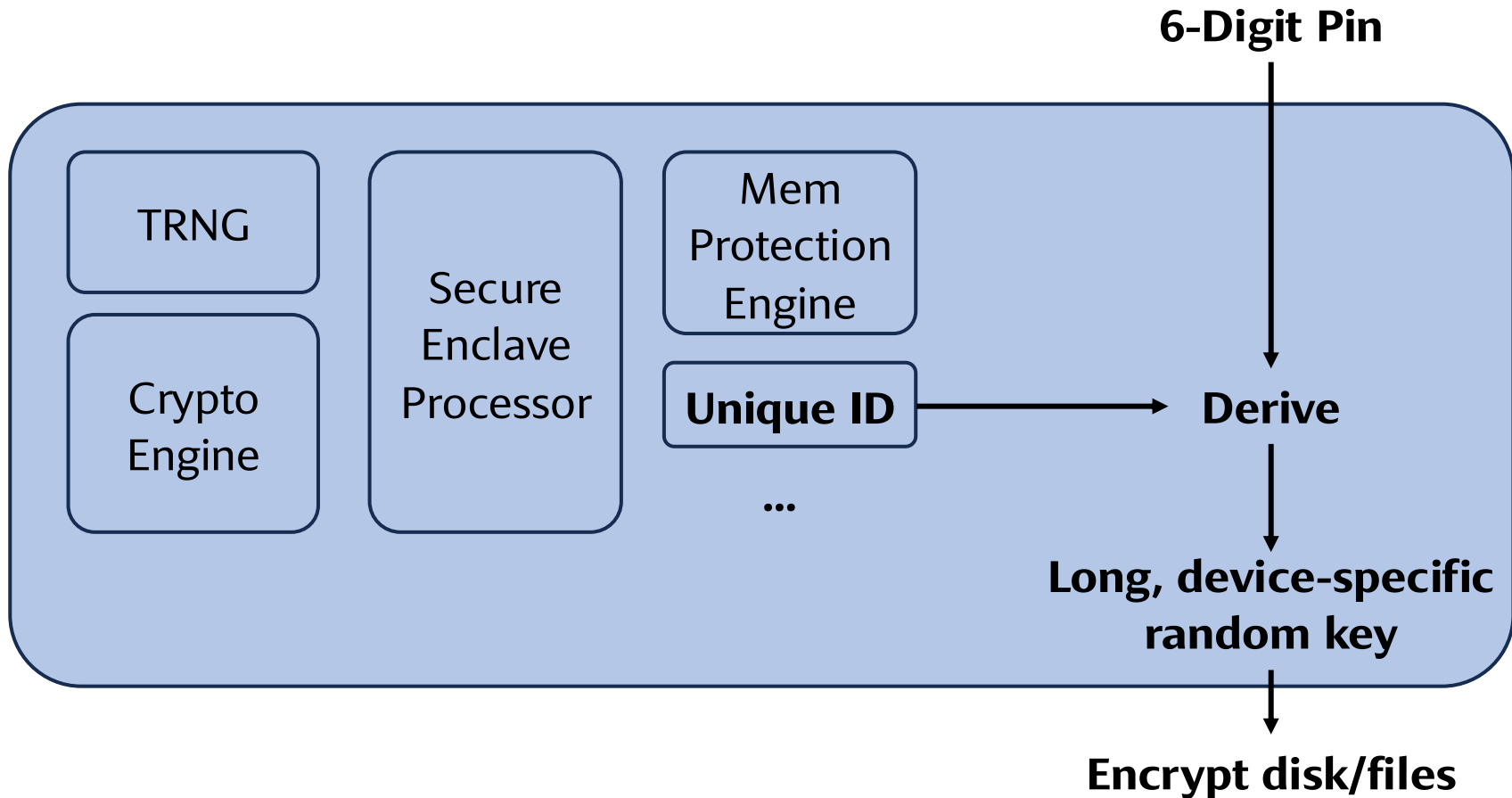
Attempt 1: Brute force the 6-digit pin → 1 million possibilities

Defender: The device rate limits attempts

Attempt 2: Make a copy of the entire disk, brute force on an attacker-controlled machine

Defender: ...

Example: Passcode-Protected Storage



On this device:

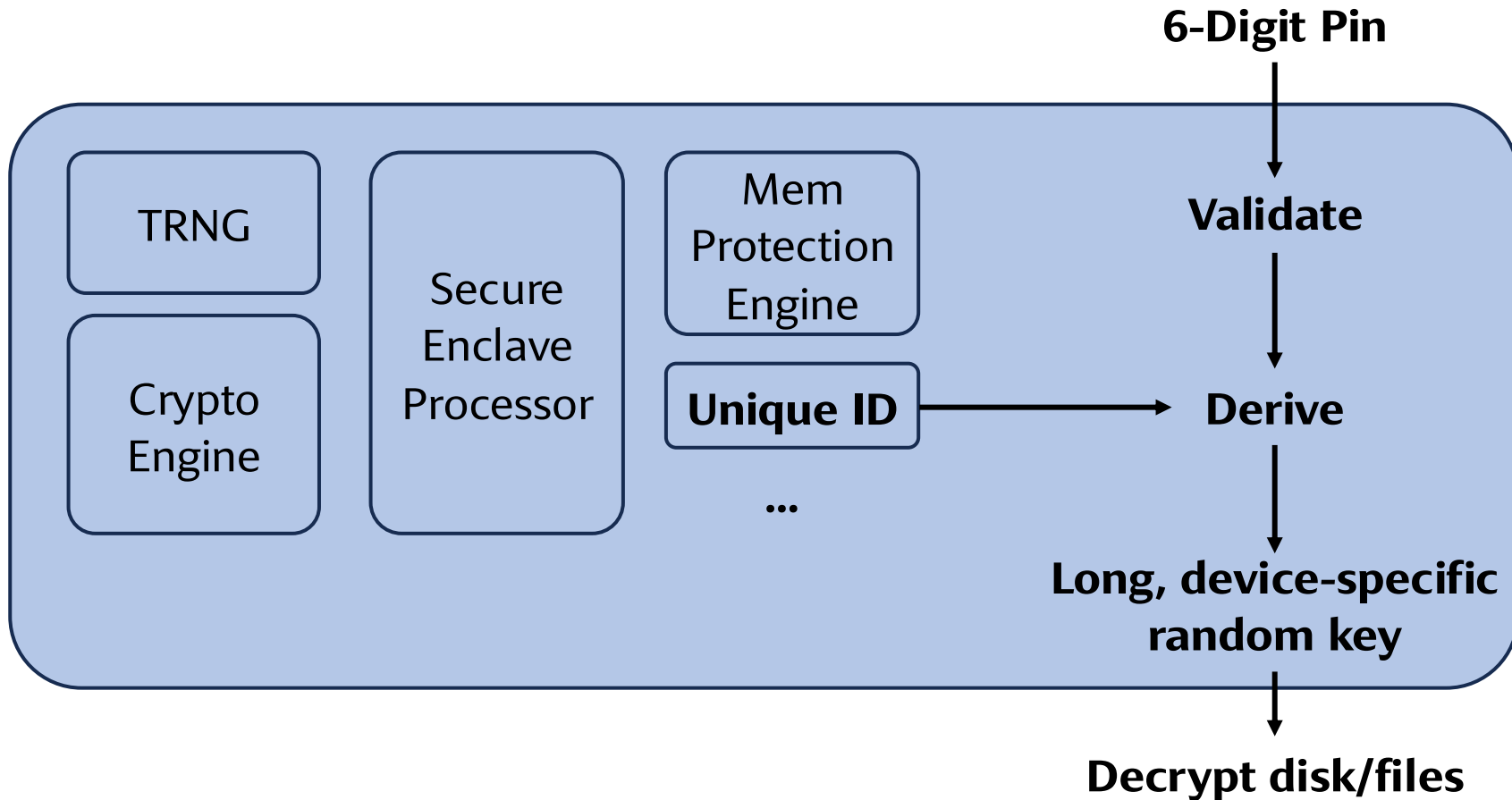
- 1M possibilities
⇒ Rate limited

On other devices:

- The original UID is inaccessible
⇒ Need to brute force the long random key

*Again, just an illustration, the real scheme is more complex

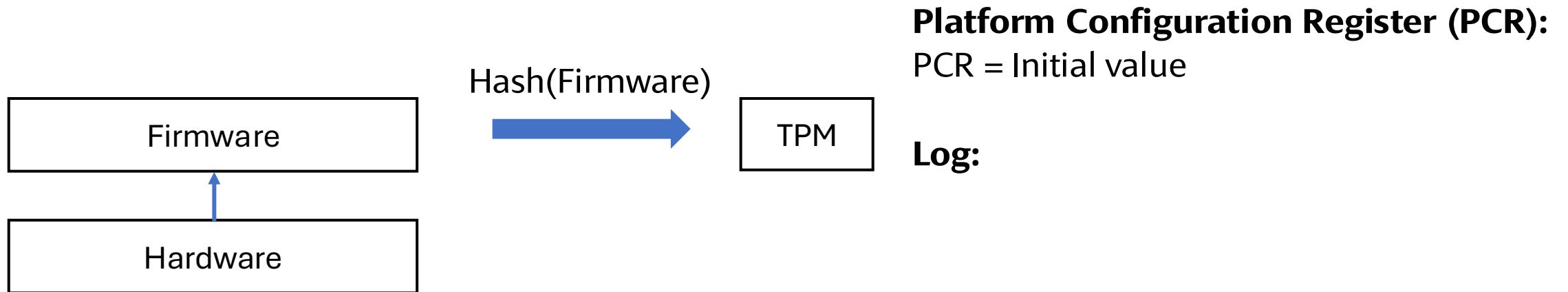
Decryption Process



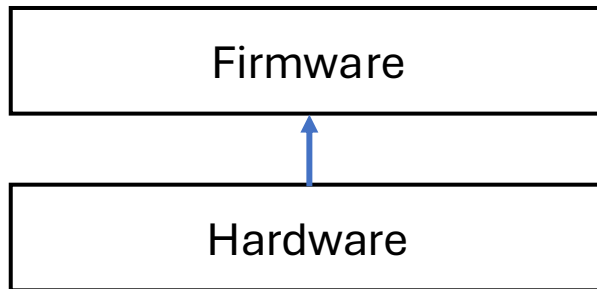
- **Incorrect:**
 - ⇒ Increment a counter
 - ⇒ More than 10 failures?
Erase the key!
- **Correct:**
 - ⇒ Reset the counter to 0
 - ⇒ Decrypt

*Again, just an illustration, the real scheme is more complex

Measured Boot with Trusted-Platform Module (TPM)



Measured Boot with Trusted-Platform Module (TPM)



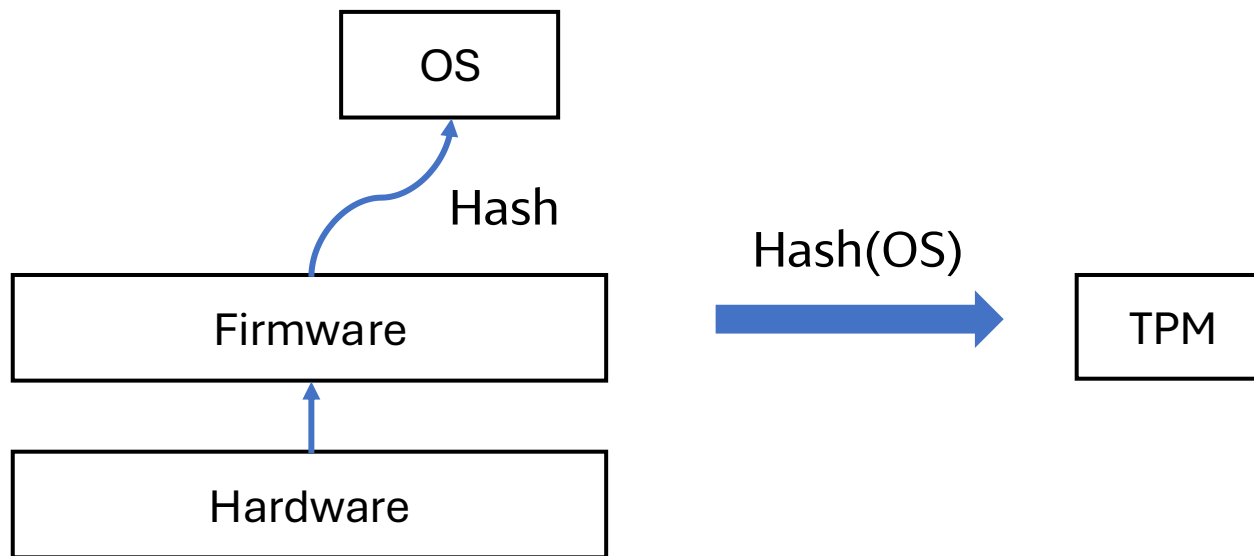
Platform Configuration Register (PCR):

$$\text{PCR} = \text{Hash}(\text{PCR}_{\text{old}} \parallel \text{Hash}(\text{Firmware}))$$

Log:

Measured firmware, hash = xxxxxx

Measured Boot with Trusted-Platform Module (TPM)



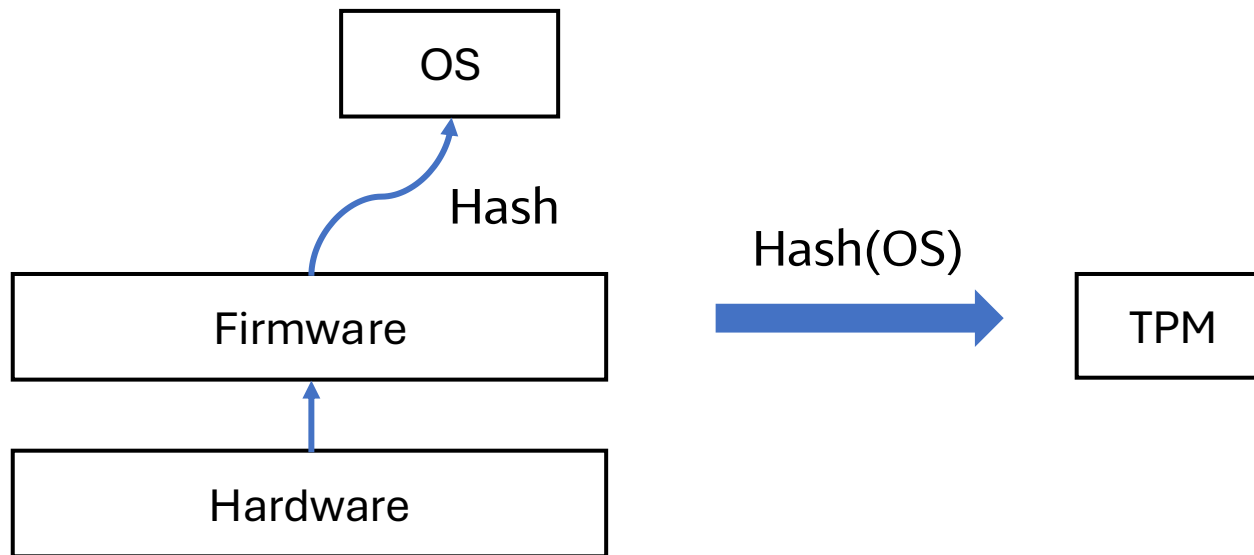
Platform Configuration Register (PCR):

$PCR = \text{Hash}(PCR_{old} || \text{Hash}(\text{Firmware}))$

Log:

Measured firmware, hash = xxxxxx

Measured Boot with Trusted-Platform Module (TPM)



Platform Configuration Register (PCR):

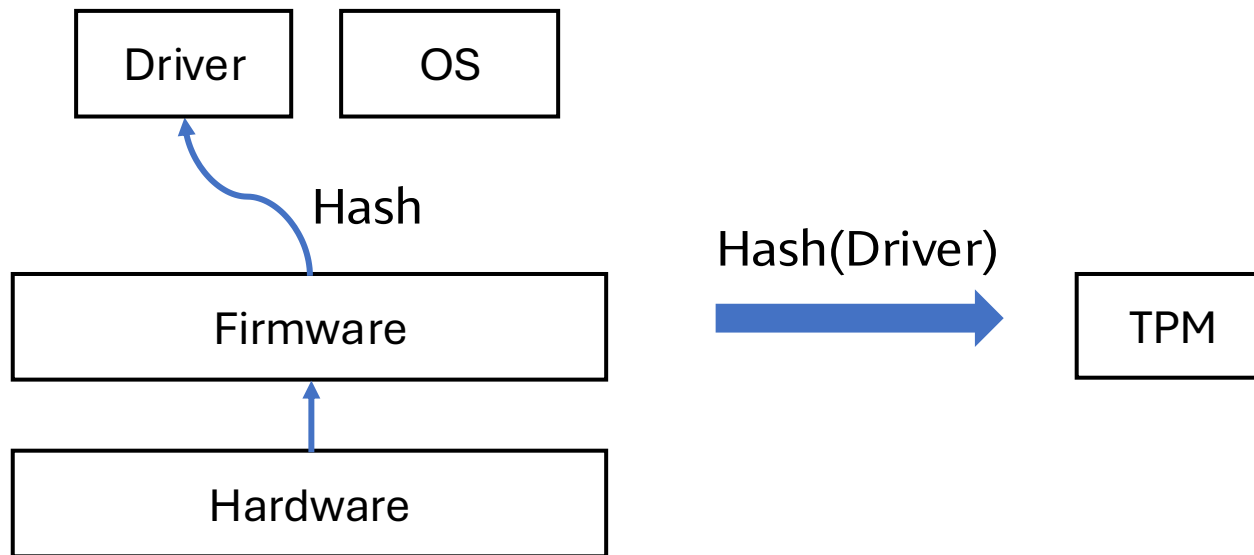
$PCR = \text{Hash}(PCR_{\text{old}} || \text{Hash}(\text{OS}))$

Log:

Measured firmware, hash = xxxxxx

Measured OS, hash = yyyyyy

Measured Boot with Trusted-Platform Module (TPM)



Platform Configuration Register (PCR):

$PCR = \text{Hash}(PCR_{old} || \text{Hash}(\text{Driver}))$

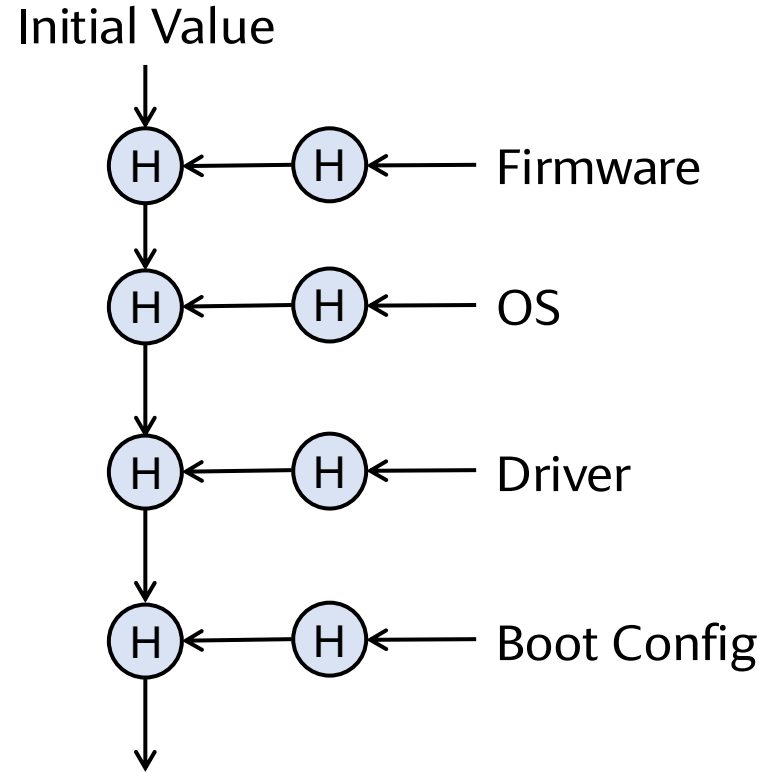
Log:

Measured firmware, hash = xxxxxx

Measured OS, hash = yyyyyy

Measured driver, hash = zzzzzz

Measured Boot with Trusted-Platform Module (TPM)



The final measurement summarizes the system's state of interest

Example use cases:

- Prove to a remote party that the system is booted with the expected measured configuration
- Automatically unlock the drive if the measurement agrees with the expected value

Final Measurement in PCR*

*Simplified here. Real TPM has multiple PCRs for different categories of measurements